Լաբորատոր աշխատանք N15

1. Աշխատանքի նպատակը

- Ծրագրավորման մեջ յուրացնել դինամիկ հիշողության կառավարման սկզբունքները և **Rule of Three** (Երեքի կանոն) հայեցակարգը C++ լեզվով (պարամետրերով կոնստրուկտոր, պատճենող կոնստրուկտոր, վերագրման օպերատոր և դեստրուկտոր):
- Գործնականումիրականացնել սովորական դիֆերենցիալ հավասարումների (Կոշիի խնդրի) մոտավոր թվային լուծումը՝ **Էյլերի մեթոդի** միջոցով:
- Ապահովել մուտքային տվյալների սահմանափակումների ստուգումը (մասնավորապես՝ լոգարիթմների և մաթեմատիկական ֆունկցիաների սահմանման տիրույթը):

2. Աշխատանքի պահանջը (Խնդիր N15)

- **Տրված է**. Սովորական դիֆերենցիալ հավասարում՝ $y' = \frac{1}{x} - \frac{y}{x}$, սկզբնական պայմանով՝ $y(1) = 2$:
- **Քայլ և քանակ**. $h = 0.05, N = 20$ (հաշվել 21 արժեք՝ $i = 0, 1, \dots, 20$):
- **Պահանջվում է**.
    1. Ստեղծել համապատասխան դաս դինամիկ զանգվածներով:
    2. Կատարել հաշվարկները Էյլերի մեթոդով:
    3. Արդյունքները արտածել աղյուսակային տեսքով:
    4. Հաշվել և արտածել ամփոփիչ ցուցանիշը՝ քանի արժեք է գտնվում $[1, 2]$ միջակայքում (ներառյալ սահմանները):

3. Ընդհանուր պահանջներ

- Յուրաքանչյուր խնդրի համար ստեղծված է առանձին դաս (EulerCauchyProblem), որի private բաժնում հայտարարված են double* դինամիկ զանգվածները (x և y արժեքների համար) և դրանց չափը (n):
- Ամբողջությամբ պահպանված է **Rule of Three**-ն (խորքային պատճենում deep copy-ի միջոցով, որպեսզի հիշողության կրկնակի ջնջման խնդիրներ չառաջանան):
- Իրականացված է սահմանման տիրույթի ստուգում ($x > 0$), ինչպես նաև սխալ մուտքագրումների կամ անվավեր արժեքների դեպքում բացառությունների (std::invalid_argument , std::runtime_error) նետում և հասկանալի հաղորդագրությունների արտածում:

4. Տեսական մաս

Սովորական դիֆերենցիալ հավասարումների համար **Կոշիի խնդիրը** կայանում է այնպիսի $y(x)$ ֆունկցիայի որոնման մեջ, որը բավարարում է տրված դիֆերենցիալ հավասարմանը և նախնական պայմանին՝ $y(x_0) = y_0$:

Առաջին կարգի դիֆերենցիալ հավասարումների համար՝

$$y' = f(x, y), y(x_0) = y_0$$

Ամենապարզ թվային մեթոդներից է **Էյլերի մեթոդը**, որի բանաձևն է.

$$y_{i+1} = y_i + h \cdot f(x_i, y_i)$$

$$x_{i+1} = x_i + h$$

որտեղ՝

- h-ը քայլի մեծությունն է ($h = 0.05$),
- $f(x, y) = \frac{1}{x} - \frac{y}{x}$ տրված ֆունկցիան է,
- $i = 0, 1, 2, \dots, N$ ($N = 20$, այսինքն՝ հաշվարկվում է 21 հանգուցային կետ)։

```
#include <iostream>
#include <iomanip>
#include <stdexcept>

class EulerCauchyProblem {
private:
   int n;          // Քայլերի քանակը (N = 20, կետերի քանակը = N + 1 = 21)
   double h;       // Քայլի մեծությունը (0.05)
   double x0;      // Սկզբնական x (1.0)
   double y0;      // Սկզբնական y (2.0)
   double* x;      // x արժեքների դինամիկ զանգված
   double* y;      // y արժեքների դինամիկ զանգված

public:
   // 1. Պարամետրերով կոնստրուկտոր
   EulerCauchyProblem(int steps = 20, double stepSize = 0.05, double startX = 1.0, double
startY = 2.0)
       : n(steps), h(stepSize), x0(startX), y0(startY) {

       if (n <= 0 || h <= 0) {
           throw std::invalid_argument("Սխալ: n և h պետք է լինեն դրական թվեր:");
       }
       if (x0 <= 0) {
           throw std::invalid_argument("Սխալ: x0-ը պետք է մեծ լինի 0-ից (լոգարիթմական
սահմանման տիրույթ):");
       }
```

```
    x = new double[n + 1];
    y = new double[n + 1];
}

// 2. Պատճենող կոնստրուկտոր (Rule of Three - Խորքային պատճենում)
EulerCauchyProblem(const EulerCauchyProblem& other)
    : n(other.n), h(other.h), x0(other.x0), y0(other.y0) {

    x = new double[n + 1];
    y = new double[n + 1];
    for (int i = 0; i <= n; ++i) {
        x[i] = other.x[i];
        y[i] = other.y[i];
    }
}

// 3. Վերագրման օպերատոր (Rule of Three - Խորքային պատճենում)
EulerCauchyProblem& operator=(const EulerCauchyProblem& other) {
    if (this == &other) {
        return *this; // Ինքնավերագրման ստուգում
    }

    // Հին հիշողության ազատում
    delete[] x;
    delete[] y;

    n = other.n;
    h = other.h;
    x0 = other.x0;
    y0 = other.y0;

    // Նոր հիշողության հատկացում և պատճենում
    x = new double[n + 1];
    y = new double[n + 1];
    for (int i = 0; i <= n; ++i) {
        x[i] = other.x[i];
        y[i] = other.y[i];
    }

    return *this;
}

// 4. Դեստրուկտոր (Rule of Three)
~EulerCauchyProblem() {
```

```
        delete[] x;
        delete[] y;
    }

    // Աջ կողմի ֆունկցիան՝ f(x, y) = 1/x - y/x
    double f(double px, double py) const {
        if (px <= 0) {
            throw std::runtime_error("Սխալ: x-ը հասել է կամ փոքր է 0-ից (խախտվում է
սահմանման տիրույթը):");
        }
        return (1.0 / px) - (py / px);
    }

    // Հաշվարկների իրականացման մեթոդ (Էյլերի մեթոդ)
    void calculate() {
        x[0] = x0;
        y[0] = y0;

        for (int i = 0; i < n; ++i) {
            // Էյլերի բանաձևը
            y[i+1] = y[i] + h * f(x[i], y[i]);
            x[i+1] = x[i] + h;
        }
    }

    // Արդյունքների ցուցադրում աղյուսակով
    void displayTable() const {
        std::cout << "----------------------------------------\n";
        std::cout << std::setw(6) << "i"
                  << std::setw(14) << "x[i]"
                  << std::setw(14) << "y[i]" << "\n";
        std::cout << "----------------------------------------\n";
        for (int i = 0; i <= n; ++i) {
            std::cout << std::setw(6) << i
                      << std::setw(14) << std::fixed << std::setprecision(4) << x[i]
                      << std::setw(14) << std::fixed << std::setprecision(4) << y[i] << "\n";
        }
        std::cout << "----------------------------------------\n";
    }

    // Ամփոփիչ ցուցանիշի հաշվարկ (քանի արժեք է գտնվում [minVal, maxVal]
միջակայքում)
    int countValuesInRange(double minVal, double maxVal) const {
        int count = 0;
```

```
        for (int i = 0; i <= n; ++i) {
            if (y[i] >= minVal && y[i] <= maxVal) {
                count++;
            }
        }
        return count;
    }
};

int main() {
    try {
        // Հիմնական օբյեկտի ստեղծում (Խնդիր N15)
        EulerCauchyProblem problem1(20, 0.05, 1.0, 2.0);
        problem1.calculate();

        std::cout << "=== ԽՆԴԻՐ N15: ԿՈՇԻԻ ԽՆԴԻՐ (ԷՅԼԵՐԻ ՄԵԹՈԴ) ===\n";
        problem1.displayTable();

        int inRangeCount = problem1.countValuesInRange(1.0, 2.0);
        std::cout << "Ամփոփիչ ցուցանիշ (y-ի արժեքները [1, 2] միջակայքում): " <<
inRangeCount << " հատ\n\n";

        // Երկու անկախ օբյեկտների և խորքային պատճենման ստուգում
        std::cout << "=== ԵՐԿՈՒ ԱՆԿԱԽ ՕԲՅԵԿՏՆԵՐԻ ԵՎ ԽՈՐՔԱՅԻՆ ՊԱՏՃԵՆՄԱՆ
ՍՏՈՒԳՈՒՄ ===\n";
        EulerCauchyProblem problem2 = problem1; // Պատճենող կոնստրուկտորի կանչ

        std::cout << "problem2 (ստեղծված պատճենող կոնստրուկտորով) աղյուսակը:\n";
        problem2.displayTable();

        // Վերագրման օպերատորի ստուգում
        EulerCauchyProblem problem3;
        problem3 = problem1;
        std::cout << "problem3 (ստեղծված վերագրման օպերատորով) հաջողությամբ
պատճենվեց:\n";

    } catch (const std::exception& e) {
        std::cerr << "Բացառություն (Սխալ): " << e.what() << '\n';
    }

    return 0;
}
```